Kian: A Fully Offline Voice Assistant for Children's Education

Aaron Schweiger, 2026

Abstract

Kian is an open-source, fully offline voice assistant designed for children's education [1]. Running entirely on an NVIDIA Jetson Orin, it requires no internet connection and no parent-managed accounts—children interact through speech alone. A multi-stage content filtering pipeline reduces exposure to harmful material, while a heuristic LaTeX-to-speech post-processing system makes audio-based mathematical question answering feasible. This paper describes the architecture, design rationale, and key tradeoffs of the system.

1. Motivation

Voice assistants have become a natural interface for children to explore questions, practice concepts, and satisfy curiosity. However, existing solutions depend on cloud services, raising concerns about privacy, availability, and access control. While recent work has explored pruning and quantizing language models for on-device deployment [2], formal evaluation of complete offline voice assistant pipelines designed for children—where content safety, speech-only interaction, and age-appropriate language are first-class requirements—remains limited.

- **Privacy.** Children's voice data is sensitive. Cloud-based systems transmit raw audio to remote servers, creating risks under regulations such as COPPA. An offline system ensures that no audio or conversation data ever leaves the device.
- **Connectivity.** Many educational settings—rural schools, outdoor classrooms, vehicles, areas with unreliable infrastructure—lack consistent internet access. A fully on-device system operates anywhere.
- **Autonomy.** Children should be able to interact with an educational tool without needing a parent's phone, a managed account, or an app store. Kian is controlled entirely through speech: changing the persona, adjusting volume, setting grade level, and resetting context are all voice commands.

2. Related Work

A growing body of HCI research has examined children's interactions with voice-based conversational agents; Garg et al. [3] survey a decade of this work, identifying persistent concerns around privacy, developmental impact, and the absence of child-specific design in commercial assistants. Several open-source projects have pursued offline voice processing—notably Open Voice OS [4] (continuing the discontinued Mycroft platform), which has also been used to build offline assistants on the NVIDIA Jetson [5], and Rhasspy [6], whose author also created the Piper TTS engine [7] used in this work. These systems were originally designed around intent parsers rather than generative LLMs, though recent integrations have added LLM support. The challenge of rendering mathematics as speech has been studied since Raman's AsTeR system [8], which first converted LaTeX to structured audio; Kian's post-processor draws on this tradition, using explicit delimiters for disambiguation in the style of the MathSpeak convention.

3. System Architecture

Kian is structured as an asynchronous pipeline of five stages, all running on-device:

1. **Voice Activity Detection (VAD).** `Silero VAD` [9] (ONNX, ∼2 MB) monitors the microphone stream at 48 kHz, downsampled to 16 kHz for inference. Speech segments are extracted with a 1.2 s silence threshold. A 30-second maximum prevents unbounded memory growth from sustained noise.
2. **Speech-to-Text (STT).** `Faster-whisper` [10], an optimized reimplementation of Whisper [11], with the `base.en` model transcribes detected speech segments using int8 quantization.
3. **Language Model (LLM).** Qwen3-4B[1] [12], quantized to Q4_K_M (4-bit), runs via llama.cpp [13] with full GPU offload. The 2048-token context window is managed by dropping oldest conversation turns when history exceeds 1500 tokens. An Ollama [14] backend is also supported as an alternative runtime.
4. **Content Filtering.** A streaming n-gram detector checks each token as it is generated. On match, generation halts immediately and the conversation context is reset.
5. **Text-to-Speech (TTS).** Piper TTS [7] synthesizes speech from text fragments, streamed to a persistent ALSA output to avoid underruns. A LaTeX-to-speech post-processor converts mathematical notation before synthesis.

The pipeline is fully streaming: TTS begins speaking the first sentence while the LLM is still generating subsequent ones. Barge-in detection monitors microphone levels during playback, allowing the child to interrupt at any sentence boundary.

4. Hardware Platform

Kian targets the NVIDIA Jetson Orin Nano (8 GB unified memory). The choice is driven by two properties of interactive voice systems:

- **TTFT over throughput.** In a conversational loop, the child waits from the moment they stop speaking until the assistant begins responding. What matters is how quickly the first token arrives, not the sustained generation rate (see Section 9).
- **GPU with high memory bandwidth.** The Jetson Orin's unified memory architecture provides the bandwidth needed to keep TTFT low for quantized transformer inference, without discrete GPU memory transfer overhead.

Running headless (without a desktop environment) frees 1–1.3 GB of RAM, improving GPU layer offload. The total memory budget is approximately:

Component	Memory
OS / system (headless)	∼1.0 GB
llama.cpp + LLM (Qwen3-4B Q4_K_M)	∼2.5 GB
Safety classifier (CPU)	∼0.1 GB
STT (Whisper base.en)	∼0.2 GB
VAD (Silero) + TTS (Piper)	∼0.2 GB
KV cache (2K context)	∼0.3 GB
Headroom	∼3.2 GB

[1] Specifically the Qwen3-4B-Instruct-2507 variant, which uses non-thinking mode only and does not generate `<think>` blocks.

5. Speech-Only Control

A core design principle is that no screen, keyboard, or companion device is required. All configuration is performed through natural speech commands:

- **Identity:** "Your name is Sophie" or "I'll call you Max" changes the assistant's persona and persists the choice across sessions.
- **Grade level:** "I'm in second grade" or "I'm in grade 5" adjusts vocabulary and explanation complexity. Supports spoken numbers ("two", "second") and digits alike, for grades pre-K through 12.
- **Volume:** "Whisper", "inside voice", "speak up", and "shout" map to four volume presets (30%, 50%, 80%, 100%) controlled via PulseAudio.
- **Context:** "Let's start fresh" or "forget everything" resets conversation history. Eight minutes of silence triggers an automatic reset.
- **Voice:** "Change your voice" or "can I talk to someone else" switches to a different TTS voice.

All settings—name, grade, voice, and volume—are persisted to disk and restored on startup.

6. Adaptive Response Length

A 4B-parameter model cannot reliably judge from a single instruction when to give a brief answer versus an elaborate explanation versus a vivid story. To address this, the system prompt is rewritten before each LLM turn based on regex pattern matching of the user's input and whether Wikipedia reference material was retrieved:

- **Short** (default): 1–3 sentences. Triggered by simple questions and casual conversation (e.g., "What is the quadratic formula?").
- **Explain**: 50–150 words. Triggered by phrases such as "explain", "teach me", "tell me about", "how does", or by the presence of retrieved Wikipedia context. LaTeX math notation is permitted.
- **Story**: 100–300 words. Triggered by "tell me a story", "describe a", "imagine a", and similar creative prompts. LaTeX is suppressed to avoid injecting notation into narrative speech.

This approach moves the length decision out of the model and into deterministic pattern matching, which is more reliable at this model scale. The benchmarks in Section 11 use a fixed system prompt (short mode) to minimize output length disparities between models and different runs.

7. Content Safety

Children's content filtering requires multiple layers, since no single mechanism is sufficient:

1. **System prompt.** The LLM is instructed to behave as an educational cartoon animal speaking to a child at a specific grade level. This biases generation toward age-appropriate content.
2. **Streaming token filter.** A sliding-window n-gram matcher checks every token against a curated list of 100+ prohibited words and phrases spanning profanity, slurs, sexual content, violence, drug references, and adult/gambling websites. Detection is immediate—the response is discarded, the context is reset, and a neutral deflection is spoken.
3. **Neural safety classifier.** Each complete sentence is classified by Granite Guardian HAP-38M [15], a 38.5M-parameter RoBERTa-based binary toxicity detector purpose-built for hate, abuse, and profanity detection. The model runs as a quantized INT8 ONNX graph on CPU, avoiding GPU memory contention with the primary LLM. Inference takes approximately 30 ms per sentence. Unsafe sentences halt generation and reset the conversation context.

A throttle mechanism ensures safety checks are interleaved with TTS playback rather than executed in bursts.

The n-gram filter list is maintained as a plain text file, making it straightforward for educators and parents to review and extend. The neural classifier provides contextual understanding that rule-based filters cannot—for example, detecting abusive language composed entirely of individually benign words.

In practice, the n-gram filter reliably catches direct profanity and known slurs, while the LLM classifier catches subtler cases such as violent scenarios described in clinical language. The pipeline is weakest against indirect elicitation—a child who asks "what words are bad?" or requests a story with violent themes using age-appropriate vocabulary may receive content that neither filter flags. Misspellings and phonetic substitutions (e.g., "fck") can also evade the n-gram matcher. The system prompt biases the LLM away from these outputs, but a determined child can find gaps. We consider this an inherent limitation of rule-based and small-model classifiers operating without human oversight.

8. Mathematical Post-Processing

Audio-based educational assistants face a fundamental challenge with mathematics: spoken language is inherently sequential and ambiguous for mathematical expressions. "a plus b over c" could mean $a + \frac{b}{c}$ or $\frac{a+b}{c}$.

Kian addresses this by allowing the LLM to emit standard LaTeX notation, which is converted to spoken English before TTS synthesis. The system:

- Detects math regions via standard delimiters (`$...$`, `$$...$$`, `\(...\)`, `\[...\]`, and `\begin{equation}`).
- Converts fractions, roots, superscripts, subscripts, Greek letters, operators, set theory notation, calculus operators, and quantifiers.
- Applies disambiguation rules: compound sub-expressions (those containing addition or subtraction) are wrapped with "the quantity ... end quantity" in contexts where boundaries are ambiguous (e.g., square roots), while fractions use "over" as a natural delimiter.
- Distinguishes unary negative from binary minus: $-x$ becomes "negative x" while $a-b$ becomes "a minus b".
- Separates adjacent single-letter variables (e.g., ix becomes "i x") to prevent TTS from reading them as words, while preserving known multi-letter tokens like function names.
- Leaves prose outside math delimiters completely untouched, so punctuation like "Hey there!" is never misinterpreted as factorial.

This makes it feasible for a child to ask "what is the quadratic formula?" and hear a correct, unambiguous spoken rendering of $x = \frac{-b \pm \sqrt{b^2-4ac}}{2a}$.

9. Latency Considerations

In interactive voice applications, the relevant metric is *turn latency*: the time from when the child stops speaking to when the assistant begins responding. This decomposes as:

$$t_{\text{turn}} = t_{\text{silence}} + t_{\text{STT}} + t_{\text{TTFT}} + t_{\text{TTS,first}}$$

Table 1 shows approximate values for each component on the target hardware using Qwen3-4B under llama.cpp.

Component	Time
t_{silence} (VAD silence window)	1.2 s
t_{STT} (Whisper base.en transcription)	∼0.3 s
t_{TTFT} (LLM first token, mean)	∼0.17 s
$t_{\text{TTS,first}}$ (Piper synthesis + playback start)	∼0.1 s
Estimated turn latency	∼1.8 s

Table 1: End-to-end turn latency breakdown for Qwen3-4B (llama.cpp) on Jetson Orin Nano. Values are engineering estimates from representative runs; TTFT is the measured mean from Table 2.

The VAD silence window dominates turn latency; all other components combined add under 0.6 s. Tokens per second is not the bottleneck—even at modest generation rates, text accumulates faster than it can be spoken. The streaming architecture exploits this: TTS begins on the first sentence boundary while the LLM continues generating.

10. Wikipedia Context Augmentation

Small language models hallucinate frequently on factual questions—a serious issue for an educational tool. Kian mitigates this with a retrieval-augmented generation (RAG) pipeline backed by Simple English Wikipedia, an edition written at a reading level appropriate for children.

10.1. Offline Index

A build script (`build-wiki-db.py`) downloads the Simple Wikipedia XML dump (∼350 MB) and constructs a local SQLite database with an FTS5 full-text search index (∼170 MB). The entire index lives on-device; no network access is needed at query time.

10.2. Topic Extraction

Before each LLM turn, the child's utterance is matched against a set of 20+ regex patterns designed to extract educational topics from natural speech: "what is a black hole", "tell me about volcanoes", "who was Marie Curie", "how does photosynthesis work", and similar phrasings. Stopwords are stripped, and the remaining terms are used as an FTS5 query.

10.3. Retrieval and Injection

If the FTS5 search returns a result above a BM25 relevance threshold, the first 500 characters of the matching article are injected into the system prompt as reference material for that turn. The LLM is instructed to use the reference only if relevant and never to mention its source. When conversation turns are evicted from the context window, corresponding wiki references are also evicted, keeping the context clean.

This approach grounds the LLM's responses in factual content without requiring a larger model or internet access.

11. Model Evaluation

11.1. Methodology

We evaluated candidate models using a sequential prompt benchmark that simulates a child's conversation. Eight prompts progress from a simple greeting through factual questions to a creative storytelling request, building conversational context incrementally. Each model was assessed on three qualitative dimensions:

- **Succinctness**: Are short-answer responses concise? (Evaluated on 6 of the prompts.)
- **Complexity**: Is the storytelling response sufficiently detailed? (Evaluated on 1 creative prompt.)
- **Accuracy**: Are responses factually correct and free of hallucinations? (Evaluated across all 8 prompts.)

Models were benchmarked on the target hardware (Jetson Orin Nano, 8 GB) using two runtimes: llama-cpp-python [13] for in-process GGUF inference, and Ollama [14] as an OpenAI-compatible server with native CUDA support. The evaluated model families include Qwen3 [12], Llama 3.2 [16], Granite 3.3 [17] and 4.0 [18], Ministral-3 [19], and Nemotron-3 Nano [20].

11.2. Performance Results

All models were evaluated with a 2048-token context window across 10 benchmark runs (80 prompts per model). No temperature, top-p, or other sampling parameters were adjusted; all models used their runtime defaults. Repeating the same prompt sequence multiple times averages over two sources of variance: randomness in the model's own generated responses (due to sampling temperature), and randomness in the LLM-as-judge evaluation of qualitative scores. Table 2 summarizes latency and throughput; Table 3 summarizes qualitative response quality.

Engine	Mean TTFT (s)	s.e. (s)	p95 TTFT (s)	tok/s	RAM (GB)	
llamacpp:Granite 3.3-2B	0.093	0.005	0.199	25.4	1.6	
llamacpp:Granite 4.0 Micro IQ4	0.100	0.005	0.218	24.3	1.9	
llamacpp:Granite 4.0 Micro	0.106	0.005	0.225	18.9	2.1	
llamacpp:Granite 4.0 H-Micro	0.135	0.008	0.321	17.6	1.9	
llamacpp:Qwen3-4B	0.166	0.006	0.305	15.1	2.5	
ollama:Granite 3.3-2B	0.230	0.005	0.328	25.8	1.8	All
llamacpp:Qwen3.5-2B	0.325	0.012	0.505	25.1	1.3	
ollama:Granite 4-3B	0.358	0.005	0.471	18.5	2.4	
ollama:Qwen3-4B	0.508	0.007	0.654	15.5	3.4	
ollama:Llama 3.2-3B	0.532	0.005	0.614	19.1	2.5	
ollama:Ministral-3 3B	0.586	0.007	0.726	19.5	4.8	
ollama:Nemotron-3 Nano 4B	1.020	0.034	1.565	15.6	5.2	
ollama:Qwen3.5-2B	1.031	0.016	1.306	22.2	3.5	

models achieved 100% GPU offload.† † Kernel page cache from prior model loads (via `mmap`) was cleared between engines using `drop_caches` to ensure full offload on Jetson unified memory.

Table 2: Performance benchmarks on Jetson Orin Nano (8 GB, headless). Mean and p95 TTFT over 10 runs × 8 prompts = 80 samples per engine. All models use Q4_K_M quantization and 2048-token context. RAM is total allocation reported by the runtime including KV cache.

Among the models evaluated, Qwen3-4B under llama.cpp produced no factual errors, no verbose responses, and no weak prose across all 10 runs, while Granite 4.0 Horizon Micro matched it

Engine	Err Avg	Err Max	Chatty	Poor Prose
llamacpp:Qwen3-4B	0.0	0	0.0	0.0
llamacpp:Granite 4.0 H-Micro	0.0	0	0.0	1.1
llamacpp:Granite 3.3-2B	0.3	1	0.0	1.4
ollama:Llama 3.2-3B	0.8	3	0.3	1.2
llamacpp:Granite 4.0 Micro IQ4	1.0	1	0.0	2.0
llamacpp:Granite 4.0 Micro	1.0	1	0.0	3.0
ollama:Granite 4-3B	1.1	3	0.2	1.4
ollama:Ministral-3 3B	1.2	3	2.1	0.7
ollama:Granite 3.3-2B	1.2	4	1.1	0.9
ollama:Nemotron-3 Nano 4B	1.5	3	0.1	2.4
ollama:Qwen3-4B	1.8	3	0.8	1.3
llamacpp:Qwen3.5-2B	2.0	2	0.0	1.8
ollama:Qwen3.5-2B	3.6	5	1.5	2.3

Table 3: Qualitative response quality across 10 benchmark runs (8 prompts each). All metrics are lower-is-better: Err = factual errors per conversation, Chatty = verbose or repetitive responses (of 5), Poor Prose = weak explanatory/story responses (of 3). Evaluated by Claude Opus 4.6 (1M context).

on accuracy with only a slight weakness in prose quality (1.1). The smaller Granite 3.3-2B kept errors near zero (0.3 avg) with clean, concise output. Llama 3.2-3B was competitive at 0.8 errors on average with low verbosity (0.3 chatty), though its error ceiling was higher (max 3). Notably, inference engine choice had a pronounced effect: Qwen3-4B produced no errors under llama.cpp but degraded under its Ollama variant (1.8 errors avg, 0.8 chatty), and Granite 3.3-2B similarly worsened from 0.3 to 1.2 errors. Since the two runtimes may use different quantization implementations, prompt templates, or sampling defaults, this suggests that runtime configuration matters as much as model selection itself.

A clear accuracy–verbosity trade-off emerged across the Granite family. The full-precision **Granite 4.0 Micro** produced only 1.0 errors on average but scored worst on prose quality (3.0), generating stilted longer-form responses. Its IQ4-quantized variant matched on accuracy while improving prose to 2.0, and the 3B Ollama variant (**Granite 4-3B**) traded slightly higher error rates (1.1) for markedly better prose (1.4) and modest verbosity (0.2). **Ministral-3 3B** illustrated the opposite failure mode: reasonable accuracy (1.2) but extreme chattiness (2.1), which is particularly undesirable in a child-facing voice interface where verbose responses discourage the back-and-forth dialogue that drives learning. **Qwen3.5-2B** showed the cost of aggressive size reduction, with errors climbing from 2.0 under llama.cpp to 3.6 under Ollama—the latter hallucinating that "the fastest fish in water is the dolphin." For this use case, the results favor Qwen3-4B (llama.cpp) and Granite 4.0 Horizon Micro as the most promising options among the models tested.

11.3. Benchmark Limitations

This benchmark is a task-specific internal evaluation, not a general capability ranking. The prompt set is small (8 prompts) and English-only, covering a narrow slice of the topics a child might ask about. Qualitative scores were assigned by an LLM judge (Claude Opus 4.6) rather than human raters, which may not capture all dimensions of child-appropriateness such as tone, pacing, or emotional register. The benchmark also does not evaluate the content safety pipeline or the LaTeX-to-speech system. Results should be interpreted as a comparative signal for model selection on this hardware, not as absolute quality claims.

Engine	Representative flub
llamacpp:Qwen3-4B	—
llamacpp:Granite 4.0 H-Micro	—
llamacpp:Granite 3.3-2B	*"sailfish, being a much faster swimmer, would likely win"*
ollama:Llama 3.2-3B	*"push their bones against each other"*
llamacpp:Granite 4.0 Micro IQ4	*"sailfish would likely win because it can swim much"*
llamacpp:Granite 4.0 Micro	*"it can swim much faster than a cheetah can run"*
ollama:Granite 4-3B	*"it swims much faster than a cheetah runs"*
ollama:Ministral-3 3B	*"the faster you eat, the faster you can run"*
ollama:Granite 3.3-2B	*"Kian can sprint up to 60-70 miles per hour"*
ollama:Nemotron-3 Nano 4B	*"bluefin tuna, running over 70 miles per hour"*
ollama:Qwen3-4B	*"special** features"*
llamacpp:Qwen3.5-2B	*"almost as fast as a human sprinting"*
ollama:Qwen3.5-2B	*"The fastest fish in water is the dolphin"*

Table 4: Representative factual errors from qualitative evaluation.

12. Limitations and Future Work

- **Language support** is currently English-only across all pipeline stages.
- **Content filtering** combines rule-based n-gram matching with a neural toxicity classifier, but neither mechanism can guarantee detection of novel circumventions or subtly inappropriate content. Adversarial probing by children is a realistic threat model.
- **Mathematical rendering** handles standard LaTeX but not all edge cases (e.g., matrices, piecewise functions, or deeply nested expressions).
- **No visual output** means that some mathematical concepts (geometry, graphs) are difficult to convey through speech alone.

13. Conclusion

Kian demonstrates that a practical, fully offline educational voice assistant can run on modest edge hardware. Recent advances in small-model quality and quantization make this class of system newly feasible: a 4-bit 4B-parameter model on an 8 GB Jetson achieves sub-200 ms mean time-to-first-token with no factual errors over our internal benchmark run, while leaving enough headroom for a full speech pipeline and safety classifier.

The architectural bet is that privacy, connectivity independence, and child autonomy are worth the tradeoffs inherent in small on-device models—limited context windows, narrower world knowledge, and imperfect content filtering. For an educational voice assistant aimed at children, we believe those tradeoffs favor the on-device approach: the privacy guarantees are absolute rather than policy-dependent, the system works anywhere a power outlet exists, and a child can use it without adult mediation. The fully offline approach is likely to remain relevant regardless of how cloud-based assistants evolve, as its privacy and accessibility advantages are structural, not just a current-generation artifact.

Acknowledgments

This paper and the accompanying codebase were composed with assistance from Claude Opus 4.6 (Anthropic).

References

[1] Aaron Schweiger. Kian: A fully offline voice assistant for children's education. https://github.com/aschweig/jetson-orin-kian, 2026.

[2] Bo Liu. Comparing sparse Llama 3 and Llama 2 models for on-device AI assistants. Research Square preprint, 2024.

[3] Radhika Garg, Hua Cui, Spencer Seligson, Bo Zhang, Martin Porcheron, Leigh Clark, Benjamin R. Cowan, and Erin Beneteau. The last decade of HCI research on children and voice-based conversational agents. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, pages 1–19, 2022.

[4] OVOS Community. Open voice OS. https://www.openvoiceos.org/, 2024.

[5] NVIDIA Developer Forums. Open voice OS on Jetson Orin Nano: Offline AI assistant with LLM + TTS + STT on K3s. https://forums.developer.nvidia.com/t/open-voice-os-on-jetson-orin-nano-offline-ai-assistant-with-llm-tts-stt-on-k3s/330132, 2025.

[6] Michael Hansen. Rhasspy: Offline voice assistant toolkit. https://github.com/rhasspy/rhasspy, 2023.

[7] Michael Hansen. Piper: A fast, local neural text to speech system. https://github.com/rhasspy/piper, 2023.

[8] T. V. Raman. *Audio System for Technical Readings*. Springer, 1998.

[9] Silero Team. Silero VAD: Pre-trained voice activity detector. https://github.com/snakers4/silero-vad, 2021.

[10] SYSTRAN. Faster-whisper: Faster Whisper transcription with CTranslate2. https://github.com/SYSTRAN/faster-whisper, 2023.

[11] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, pages 28492–28518, 2023.

[12] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[13] Abetlen. llama-cpp-python: Python bindings for llama.cpp. https://github.com/abetlen/llama-cpp-python, 2023.

[14] Ollama. Ollama. https://ollama.com/, 2024.

[15] Inkit Padhi, Manish Nagireddy, Giandomenico Cornacchia, Subhajit Chaudhury, Tejaswini Pedapati, Pierre Dognin, Keerthiram Murugesan, Erik Miehling, Martín Santillán Cooper, Kieran Fraser, Giulio Zizzo, Muhammad Zaid Hameed, Mark Purcell, Michael Desmond, Qian Pan, Zahra Ashktorab, Inge Vejsbjerg, Elizabeth M. Daly, Michael Hind, Werner Geyer, Ambrish Rawat, Kush R. Varshney, and Prasanna Sattigeri. Granite guardian, 2024.

[16] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[17] Granite Team, IBM. Granite 3.0 language models. `https://github.com/ibm-granite/granite-3.0-language-models`, October 2024.

[18] IBM Research. Granite 4.0 nano language models. `https://github.com/ibm-granite/granite-4.0-nano-language-models`, 2025.

[19] Alexander H Liu, Kartik Khandelwal, Sandeep Subramanian, Victor Jouault, Abhinav Rastogi, Adrien Sadé, Alan Jeffares, Albert Jiang, Alexandre Cahill, Alexandre Gavaudan, et al. Ministral 3. *arXiv preprint arXiv:2601.08584*, 2026.

[20] NVIDIA. Nemotron-3 nano 4b. `https://ollama.com/library/nemotron-3-nano`, 2026. Accessed: 2026-04-09.